Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 4

Disciplina: Tehnici de programare

Tema: ”Introducere în Python”

A efectuat : st. gr.SI – 263 Morozovschii Carolina

A verificat : asistent universitar Rețman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Sarcina:
În cadrul lucrării de laborator se vor rezolva exercițiile propuse pe platformele Codédex – Learn Python și LearnPython.org. Exercițiile urmăresc familiarizarea cu sintaxa limbajului Python și aplicarea practică a principalelor concepte de programare.

Afișarea textului „Hi” – enunț

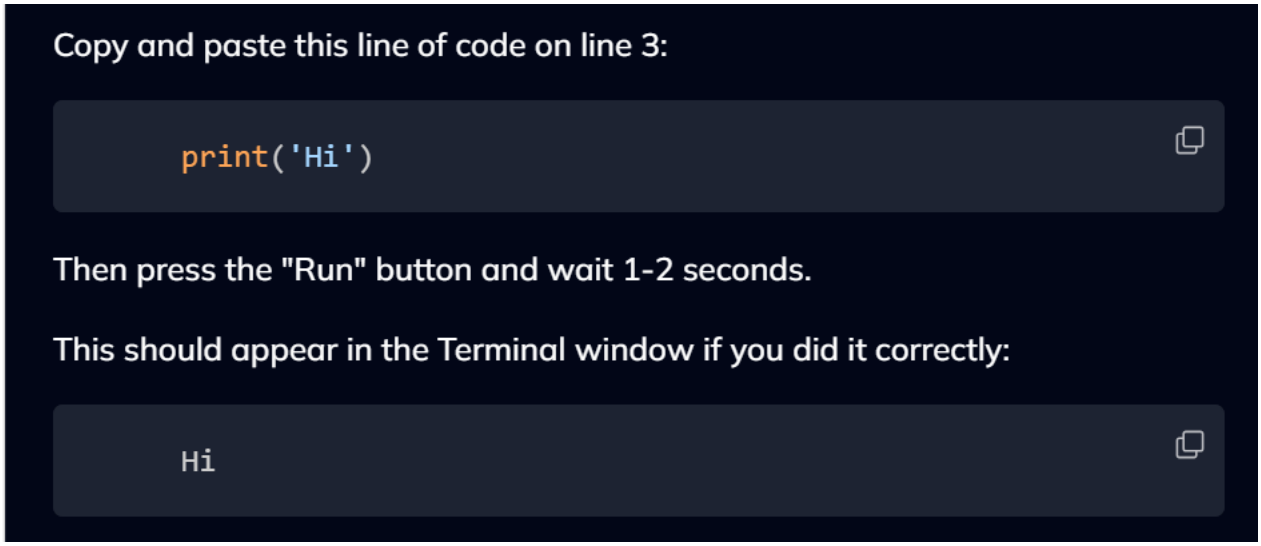


Figura 1.1 - Enunțul exercițiului pentru afișarea textului „Hi”

Afișarea textului „Hi” – rezolvare

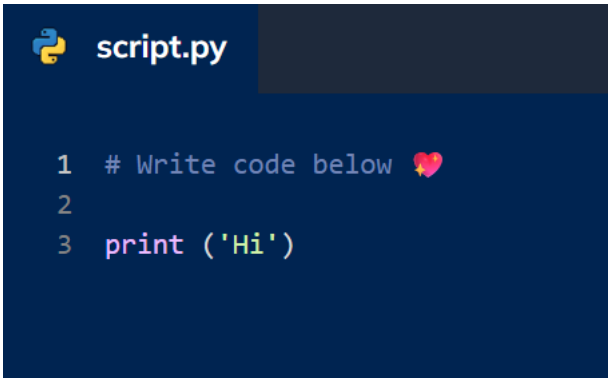


Figura 1.2 - Codul print('Hi') introdus în editorul script.py

Programul Hello World – enunț

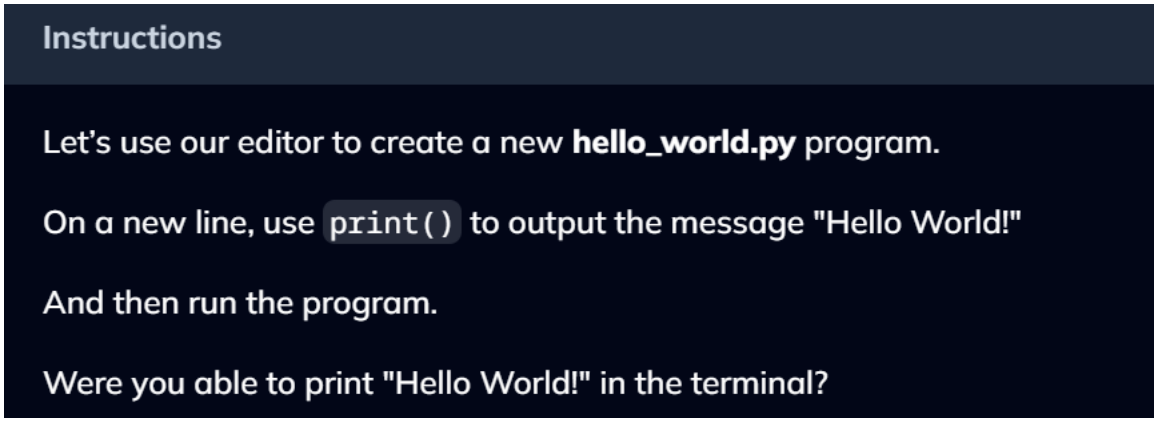


Figura 1.3 - Instrucțiunile pentru crearea programului hello_world.py

Programul Hello World – rezolvare

hello_world.py

```
1  # Write code below 💔
2  print ("Hello World!")
3
```

Figura 1.4 - Codul programului hello_world.py

Model din numere – enunț

Suppose we want the output to look exactly like this pattern below:

```
      1
    2 3
  4 5 6
7 8 9 10
```

How can you do that?

Create a **pattern.py** program that prints this pattern exactly as shown.

This will likely take some trial and error, but give it a shot!

Figura 1.5 - Enunțul exercițiului pentru programul pattern.py

Model din numere – rezolvare

pattern.py

```
1  # Write code below 💔
2  print('       1')
3  print('     2 3')
4  print('   4 5 6')
5  print('7 8 9 10')
```

Figura 1.6 - Codul programului pattern.py

Inițiale cu litere de bloc – enunț

Create an **initials.py** program that displays your initials in block letters.

First, start the program with a comment that says a fun fact about yourself.

Then, create your block letters with your initials.

For example, if your name is Dua Lipa, your initials would be D and L, and your block letters would look like this:

```
DDDD   L
D   D  L
D   D  L
D   D  L
D   D  L
D   D  L
DDDD   LLLLL
```

Figura 1.7 - Enunțul exercițiului pentru programul initials.py

Inițiale cu litere de bloc – rezolvare

initials.py

```
1  # Write code below 💖
2  #i love cars
3  print('MMMM  C')
4  print('M   M C')
5  print('M   M C')
6  print('M   M C')
7  print('M   M C')
8  print('M   M C')
9  print('MMMM  CCCCC')
```

Figura 1.8 - Codul programului initials.py

Conversia temperaturii – enunț

Create a **temperature.py** program that converts a number from Fahrenheit (°F) to Celsius (°C).

Google the current temperature of Brooklyn, NY (where Codédex is based) in °F.

Use the following formula and write it out in Python:

$$Celsius = \frac{(Fahrenheit - 32)}{1.8}$$

Print out the converted temperature. 🌡️

Figura 1.9 - Enunțul exercițiului pentru programul temperature.py

Conversia temperaturii – rezolvare

```
1  # Write code below 💖
2  fahrenheit=77;
3  celsius=(fahrenheit-32)/1.8;
4  print("Temperatura este : ",celsius)
5
```

Run

Terminal

```
Temperatura este :  25.0
```

Figura 1.10 - Codul și rezultatul conversiei din grade Fahrenheit în Celsius

Indicele de masă corporală – enunț

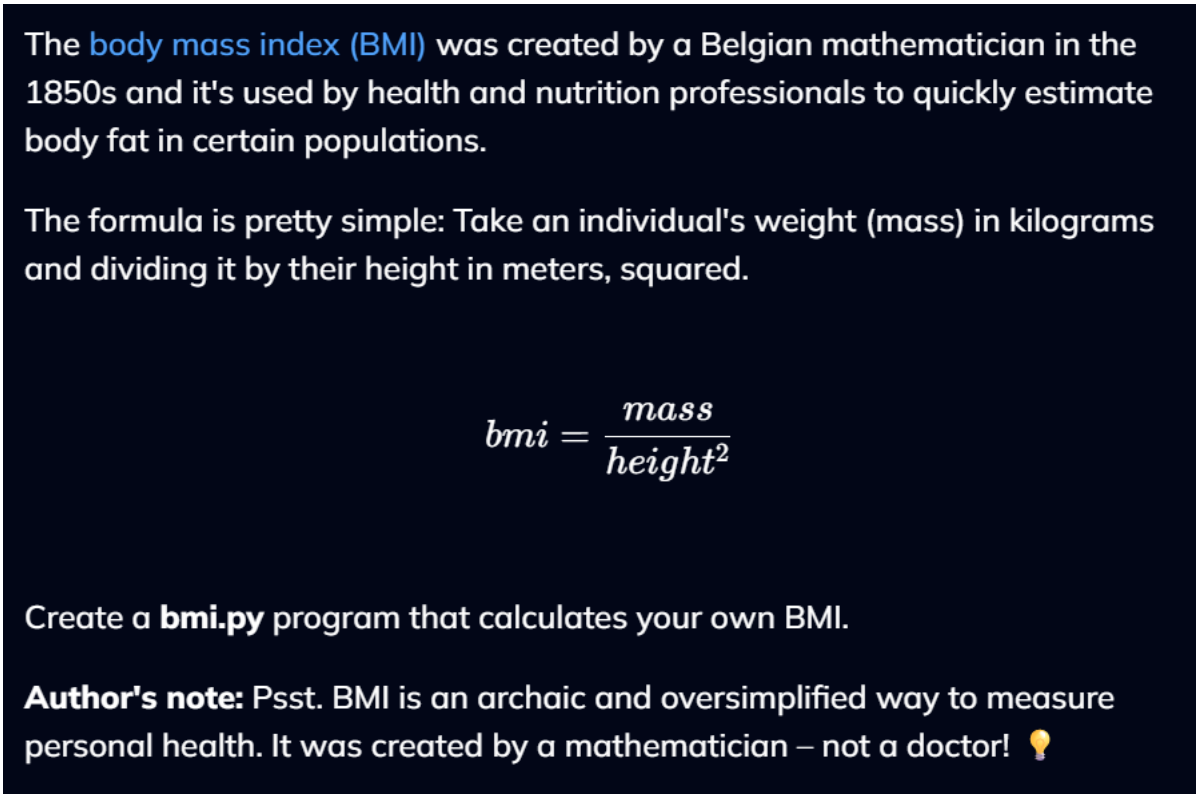


Figura 1.11 - Enunțul exercițiului pentru programul bmi.py

Indicele de masă corporală – rezolvare

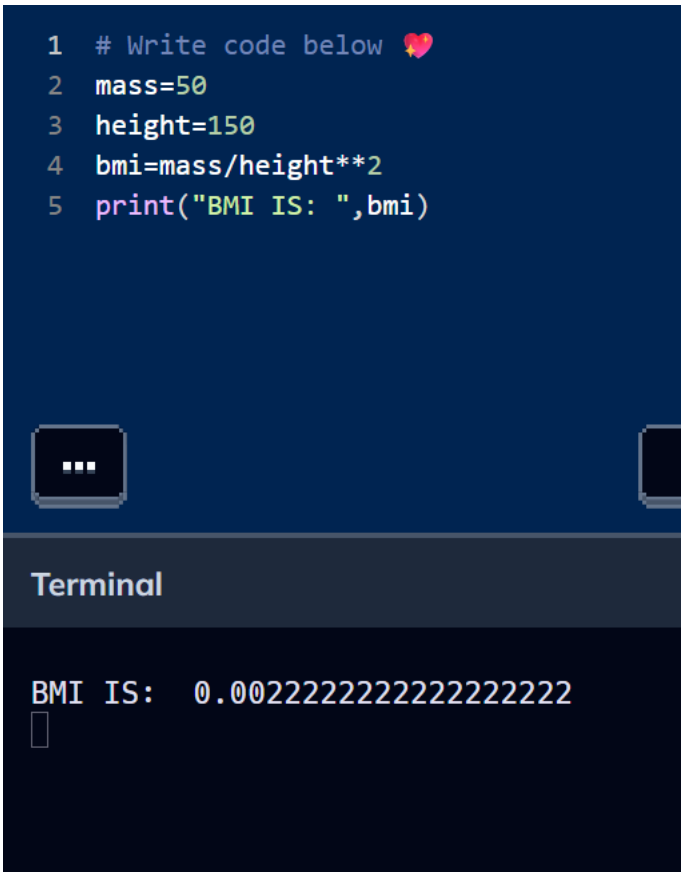


Figura 1.12 - Codul și rezultatul calculării indicelui BMI

Calculul ipotenuzei – enunț

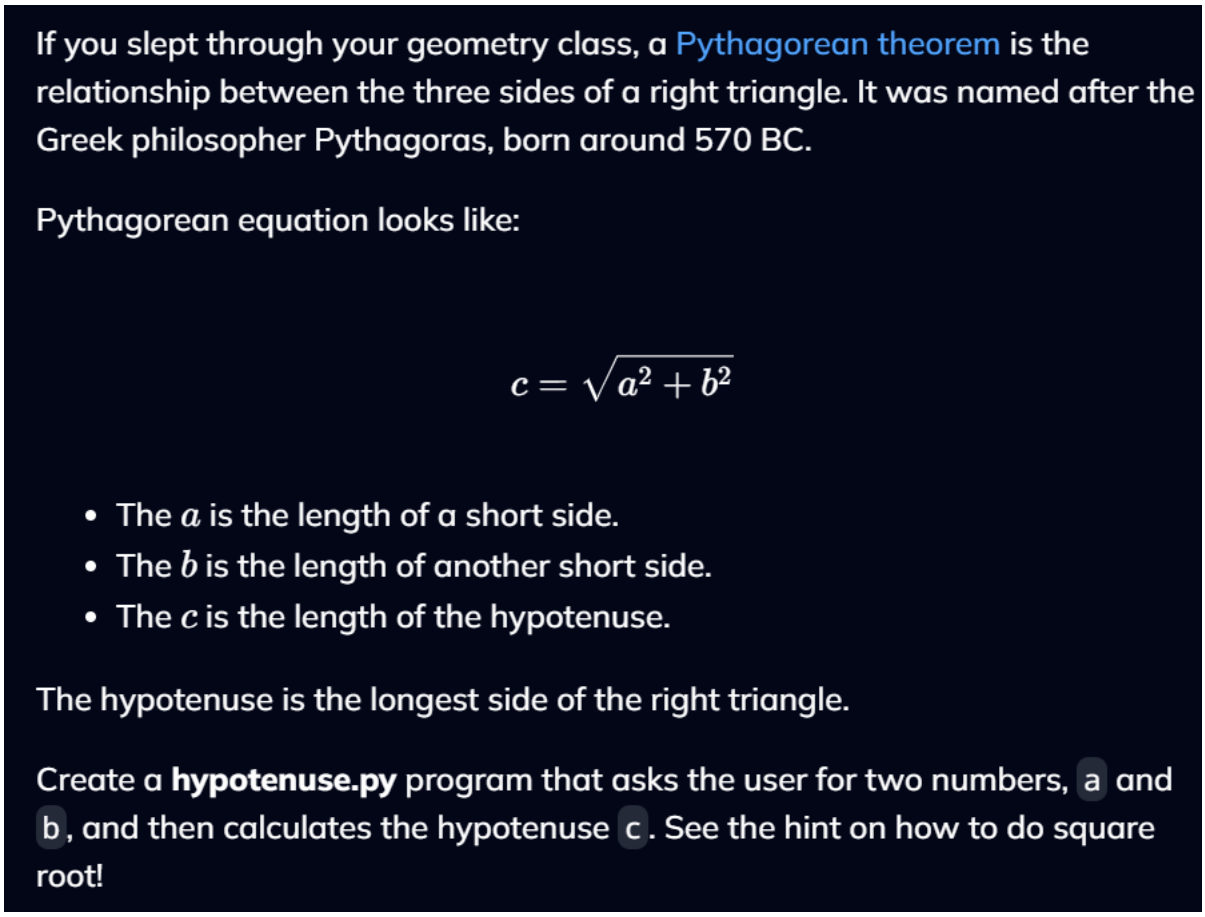


Figura 1.13 - Enunțul exercițiului pentru programul hypotenuse.py

Calculul ipotenuzei – rezolvare

```
1  # Write code below 💖
2  a=int(input('scrie a: '))
3  print(a)
4  b=int(input('scrie b: '))
5  print(b)
6  c=(a**2+b**2)**0.5
7  print(c)
```

Terminal

```
scrie a: 4
4
scrie b: 5
5
6.4031242374328485
```

Figura 1.14 - Codul și rezultatul calculării ipotenuzei

Conversia valutelor în USD – enunț

We just got home from a fun trip to South America, specifically Colombia, Peru, and Brazil. There's some leftover cash in:

- CO Colombian pesos
- PE Peruvian soles
- BR Brazilian reais

Create a **currency.py** program that asks the user for the amount they have in pesos, soles, and reais and calculates *the total* in USD.

Make sure to Google the current exchange rates for pesos, soles, and reais!

Figura 1.15 - Enunțul exercițiului pentru programul currency.py

Conversia valutelor în USD – rezolvare

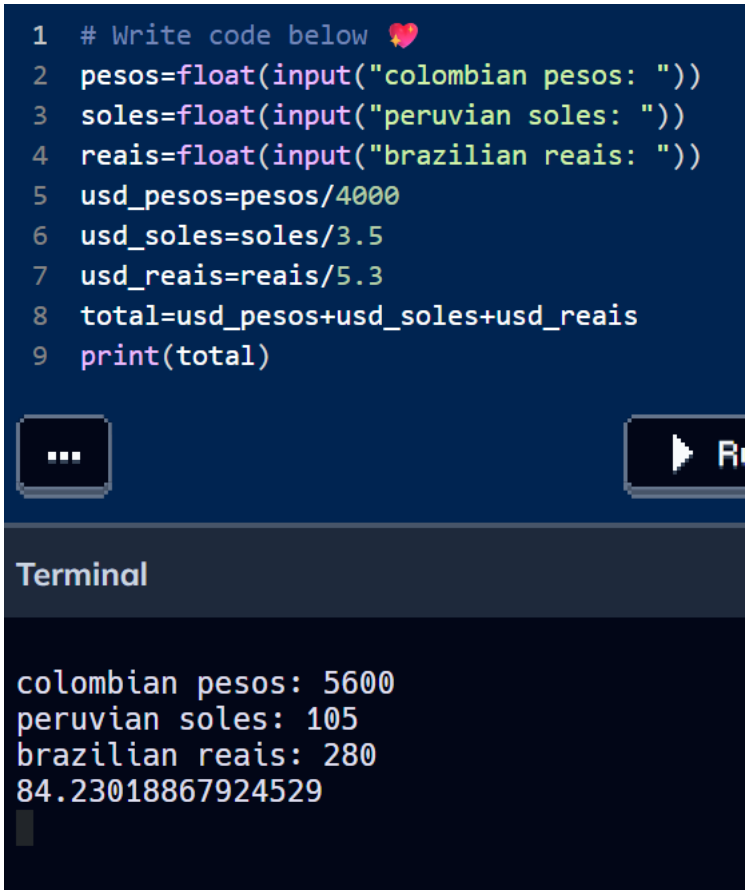


Figura 1.16 - Codul și rezultatul calculării sumei totale în USD

Afișarea mesajului „Hello, World!”

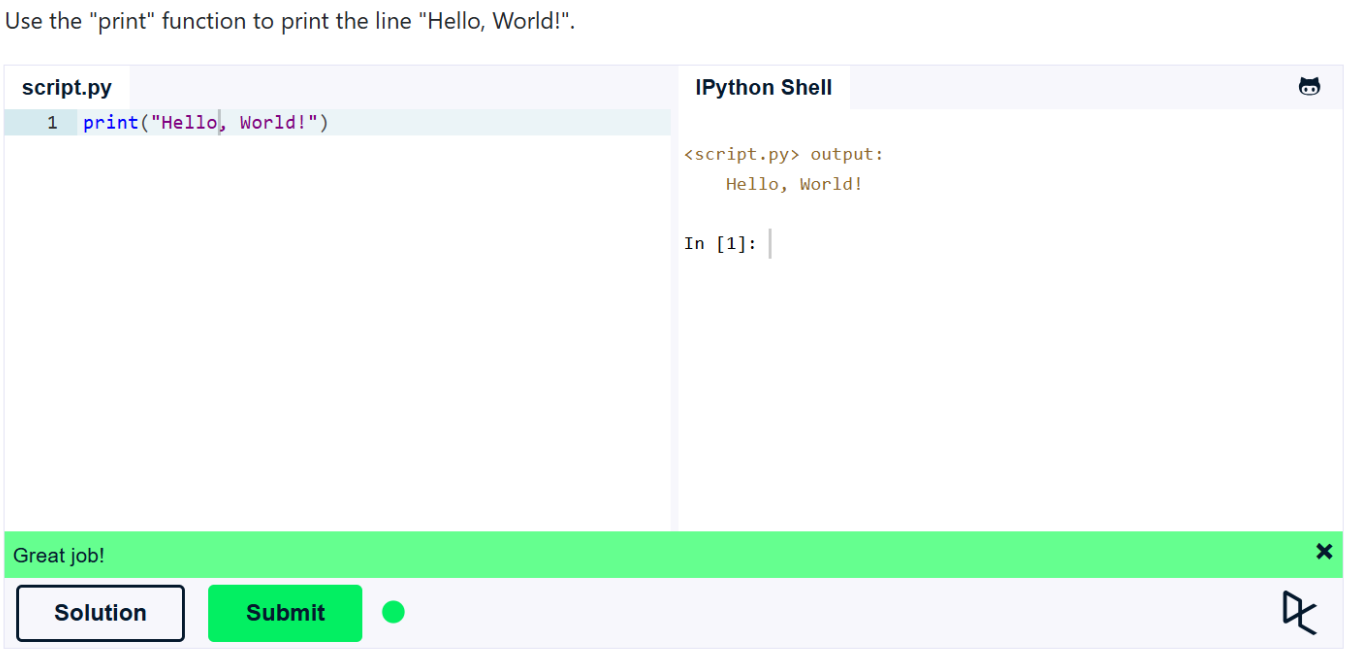


Figura 1.17 - Rezolvarea exercițiului „Hello, World!” pe LearnPython.org

Pentru acest exercițiu variantă propusă de site era “Goodbye, World!”, dar pentru a fi rezolvat am schimbat din “Goodbye” în “Hello”.

Variabile și tipuri de date

The target of this exercise is to create a string, an integer, and a floating point number. The string should be named `mystring` and should contain the word "hello". The floating point number should be named `myfloat` and should contain the number 10.0, and the integer should be named `myint` and should contain the number 20.

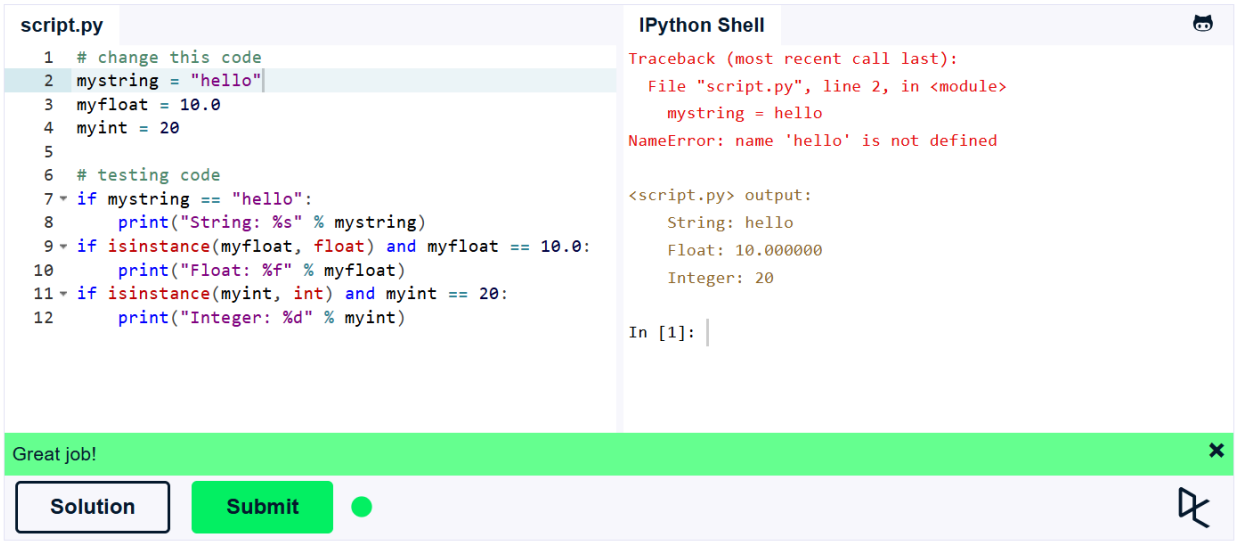


Figura 1.18 - Rezolvarea exercițiului cu variabile de tip șir, întreg și real

Aici am adăugat la mystring “hello”, la myfloat numarul 10, iar la myint numarul 20.

Liste

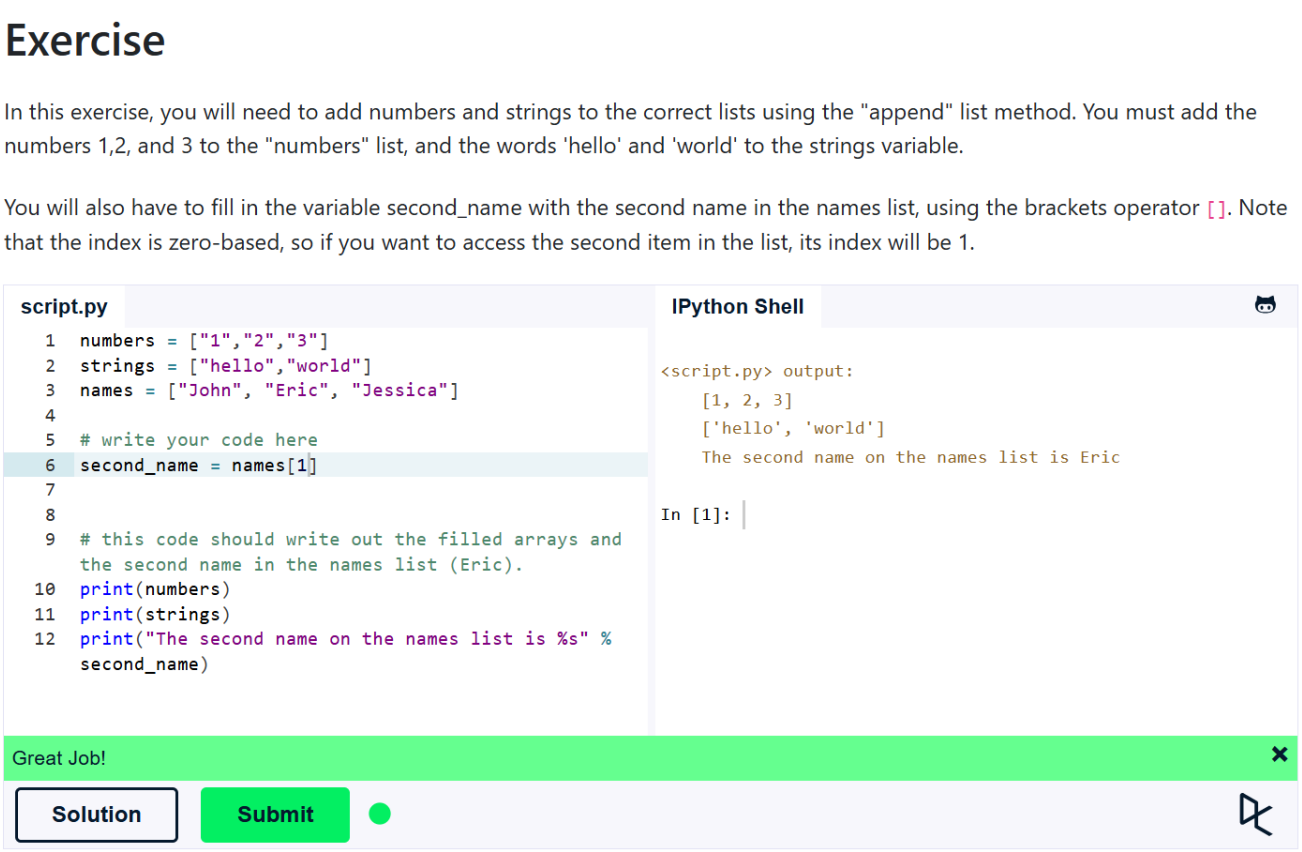


Figura 1.19 - Rezolvarea exercițiului cu liste și indexarea elementelor

Aici am adaugat numerele 1, 2 și 3 în lista numbers și cuvintele „hello” și „world” în lista strings. Am atribuit variabilei second_name al doilea element din listă, folosind indexul **1**, deoarece indexarea listelor în Python începe de la **0**.

Operatori de bază

Exercise

The target of this exercise is to create two lists called `x_list` and `y_list`, which contain 10 instances of the variables `x` and `y`, respectively. You are also required to create a list called `big_list`, which contains the variables `x` and `y`, 10 times each, by concatenating the two lists you have created.

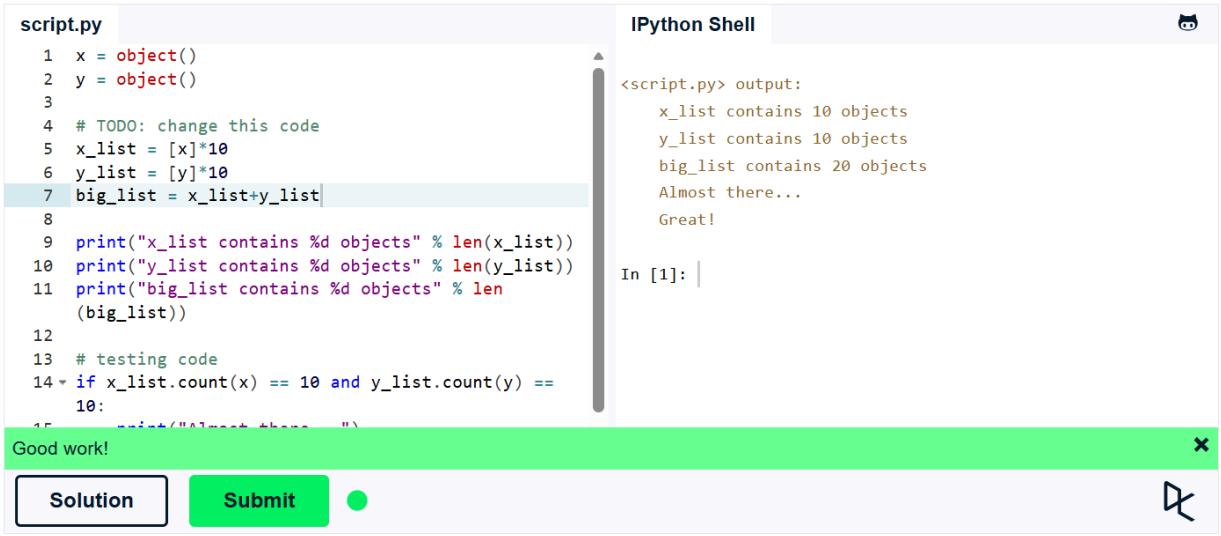


Figura 1.20 - Rezolvarea exercițiului cu operatorii * și + pentru liste

Pentru a adauga de 10 ori x si y, am utilizat operatorul de multiplicare *. Apoi pentru combinarea a x_list si y_list am utilizat operatorul + in big_list.

Formatarea șirurilor

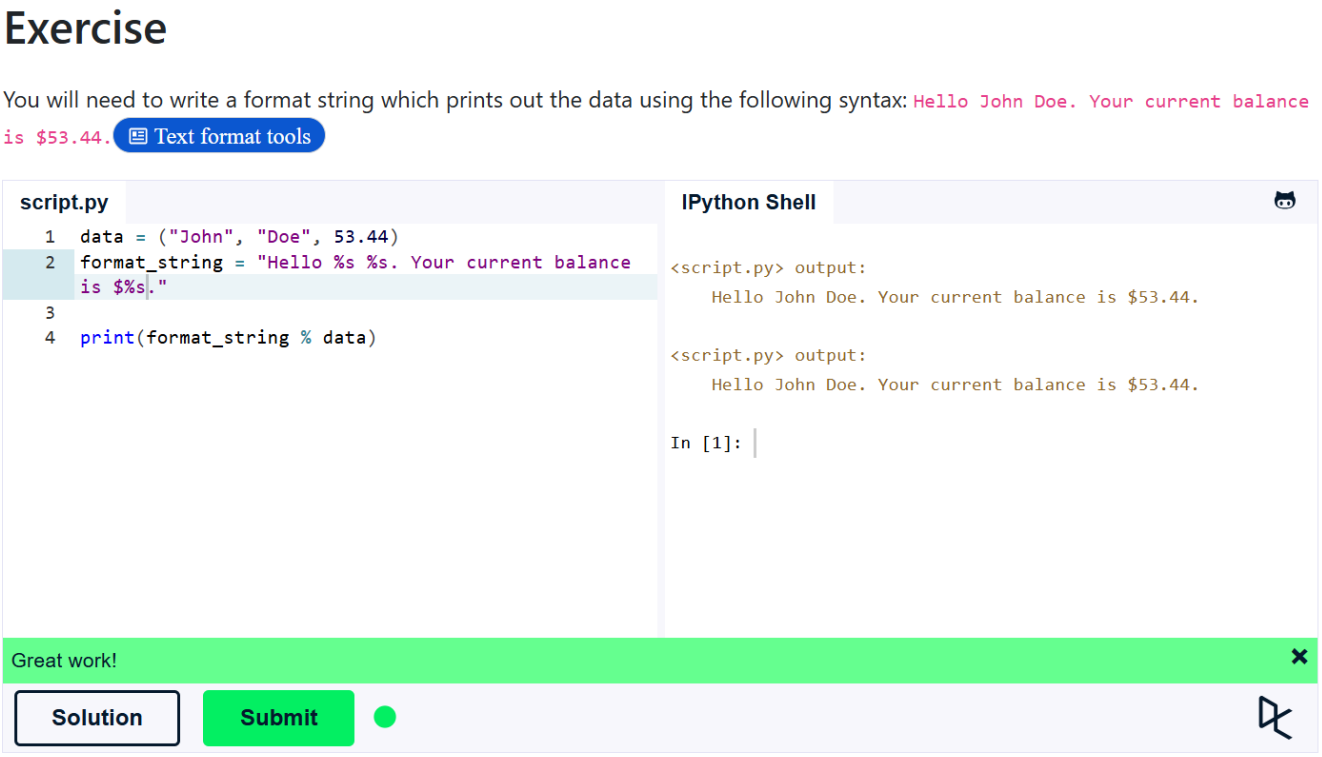


Figura 1.21 - Rezolvarea exercițiului de formatare a șirurilor de caractere

În acest exercițiu am utilizat formatarea șirurilor de caractere în Python. Datele sunt păstrate în variabila data, care conține prenumele, numele și valoarea soldului. Am utilizat specificatorul **%s** pentru introducerea fiecărei valori în șirul de caractere, inclusiv pentru valoarea numerică, aceasta fiind convertită și afișată sub formă de text. La final, cu ajutorul funcției print(), am afișat mesajul complet formatat.

Operații cu șiruri

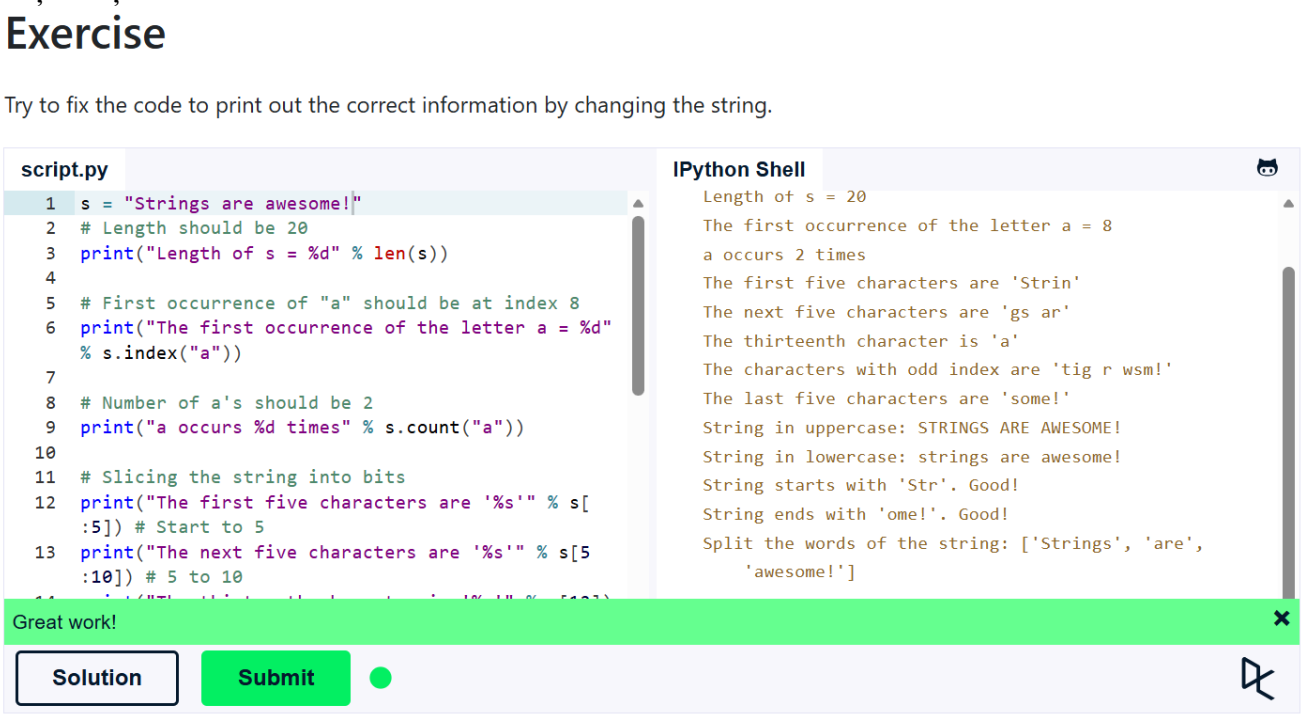


Figura 1.22 - Rezolvarea exercițiului cu operații asupra șirurilor de caractere

Am modificat șirul de caractere astfel încât programul să afișeze rezultatele cerute.

Condiții

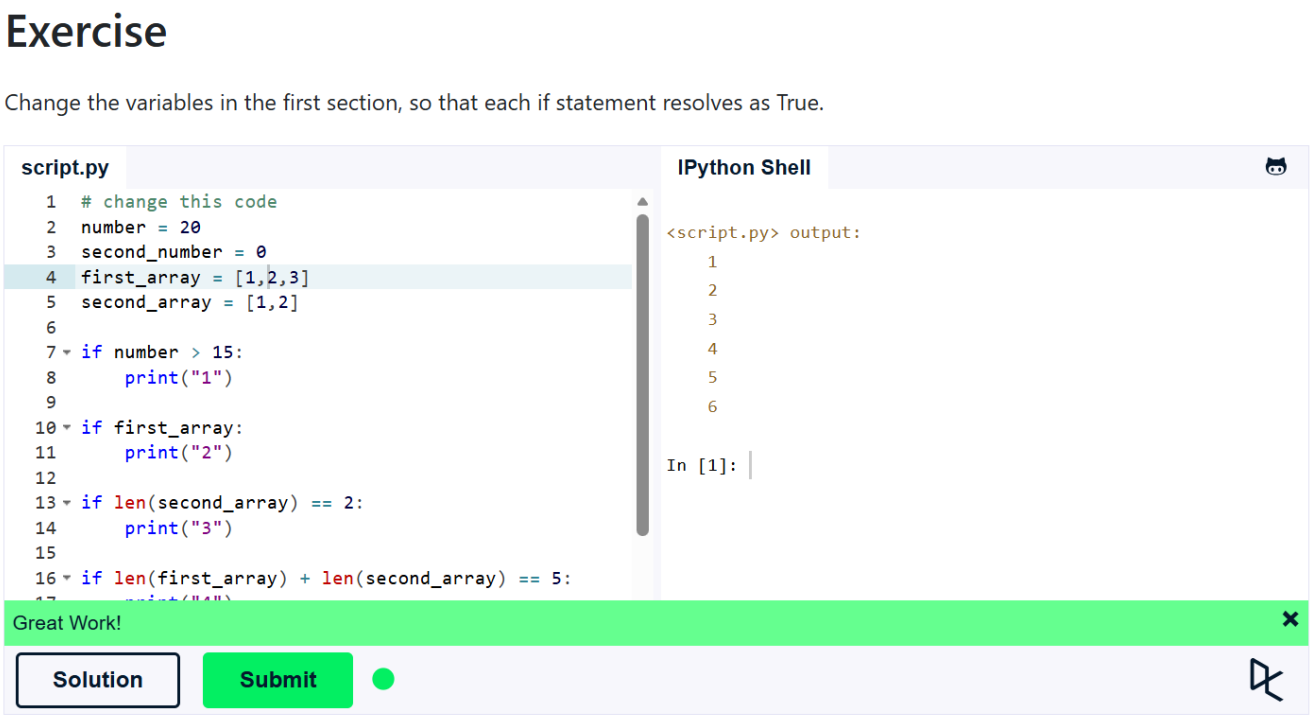


Figura 1.23 - Rezolvarea exercițiului cu condiții și instrucțiunea if

Am modificat valorile variabilelor astfel încât fiecare condiție să fie adevărată. 20 > 15; first_array nu este goală; second_array are 2 elemente; cele două liste au împreună 3 + 2 = 5 elemente; primul element din first_array este 1; iar second_number = 0, deci expresia not second_number este adevărată.

Bucle

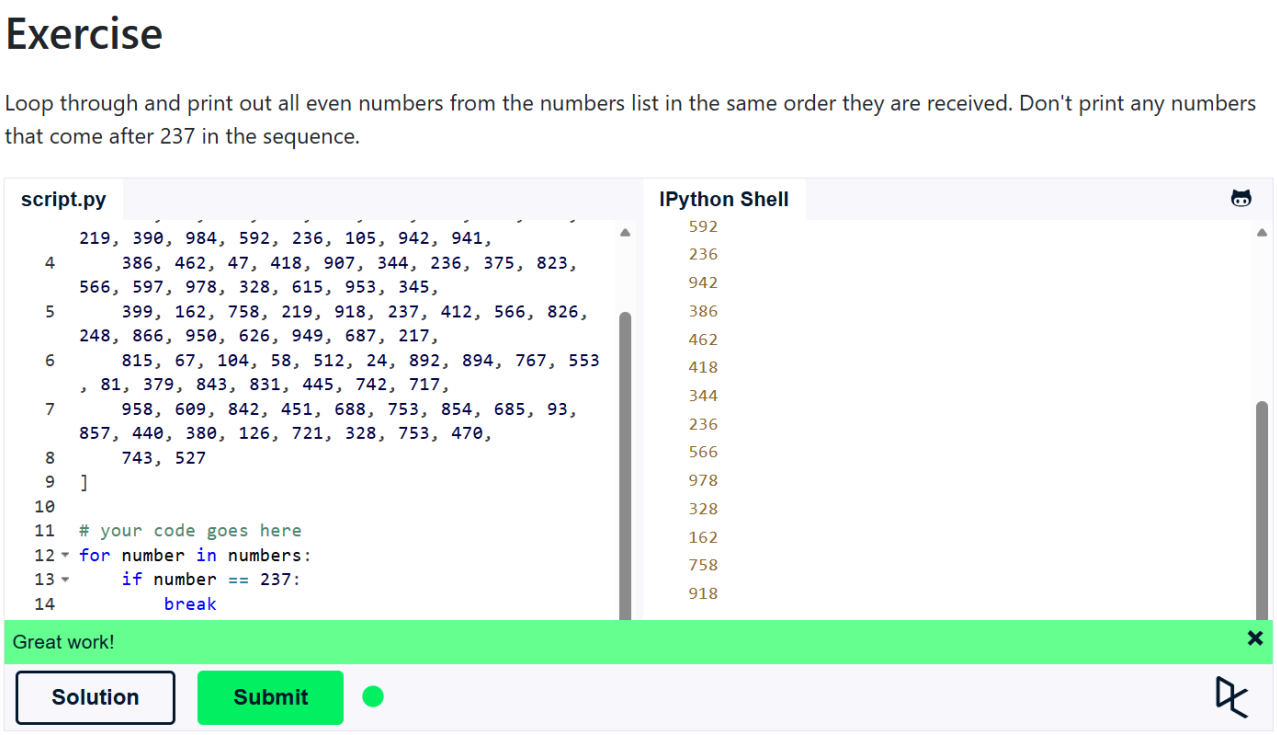


Figura 1.24 - Rezolvarea exercițiului cu bucle și instrucțiunea break

La întâlnirea valorii 237, instrucțiunea break oprește executarea buclei, astfel încât numerele aflate după această valoare nu mai sunt afișate.

Funcții

In this exercise you'll use an existing function, and while adding your own to create a fully functional program.

1. Add a function named `list_benefits()` that returns the following list of strings: "More organized code", "More readable code", "Easier code reuse", "Allowing programmers to share and connect code together" Python code blocks
2. Add a function named `build_sentence(info)` which receives a single argument containing a string and returns a sentence starting with the given string and ending with the string " is a benefit of functions!"
3. Run and see all the functions work together!

script.py

```
1  # Modify this function to return a list of strings
   as defined above
2  def list_benefits():
3      return [ "More organized code", "More readable
   code", "Easier code reuse", "Allowing programmers
   to share and connect code together"]
4
5  # Modify this function to concatenate to each
   benefit - " is a benefit of functions!"
6  def build_sentence(benefit):
7      return benefit+ " is a benefit of functions!"
8
9  def name_the_benefits_of_functions():
10     list_of_benefits = list_benefits()
11     for benefit in list_of_benefits:
12         print(build_sentence(benefit))
```

IPython Shell

```
<script.py> output:
    More organized code is a benefit of functions!
    More readable code is a benefit of functions!
    Easier code reuse is a benefit of functions!
    Allowing programmers to share and connect code together
        is a benefit of functions!

In [1]:
```

Nice work!

Solution Submit

Figura 1.25 - Rezolvarea exercițiului cu funcții

Prin intermediul buclei for, fiecare element din listă este prelucrat și apoi afișat cu funcția print().

Clase și obiecte

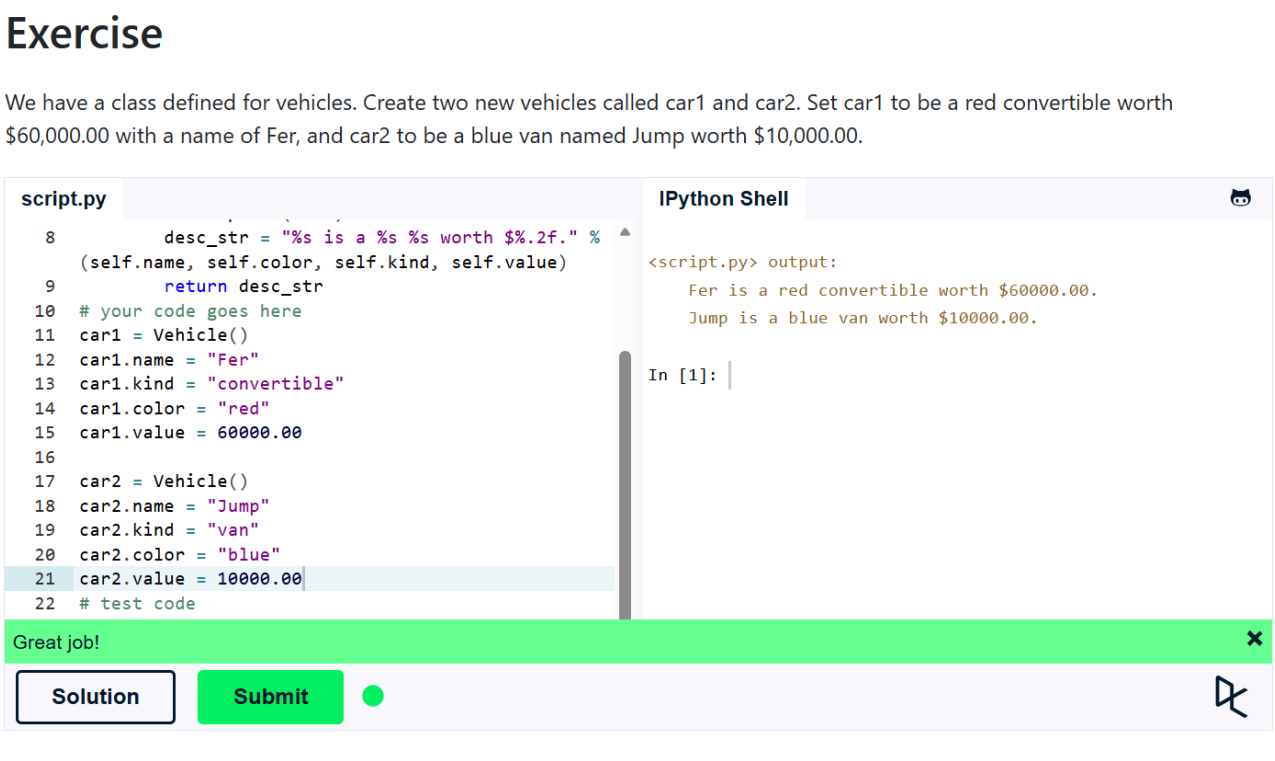


Figura 1.26 - Rezolvarea exercițiului cu clasa Vehicle și obiectele car1 și car2

Clasa Vehicle reprezintă structura generală a unui vehicul și conține caracteristici precum numele, tipul, culoarea și valoarea. Am creat două obiecte, car1 și car2, cărora le-am atribuit valori diferite pentru aceste caracteristici. Metoda description() este utilizată pentru a forma și returna descrierea fiecărui vehicul, iar funcția print() afișează rezultatele obținute.

Dicționare

Exercise

Add "Jake" to the phonebook with the phone number 938273443, and remove Jill from the phonebook.

script.py

```
phonebook = {
    "John" : 938477566,
    "Jack" : 938377264,
    "Jake" : 938273443
}
# your code goes here

# testing code
if "Jake" in phonebook:
    print("Jake is listed in the phonebook.")

if "Jill" not in phonebook:
    print("Jill is not listed in the phonebook.")
```

IPython Shell

```
<script.py> output:
    Jake is listed in the phonebook.
    Jill is not listed in the phonebook.

In [1]:
```

Nice work!

Solution Submit

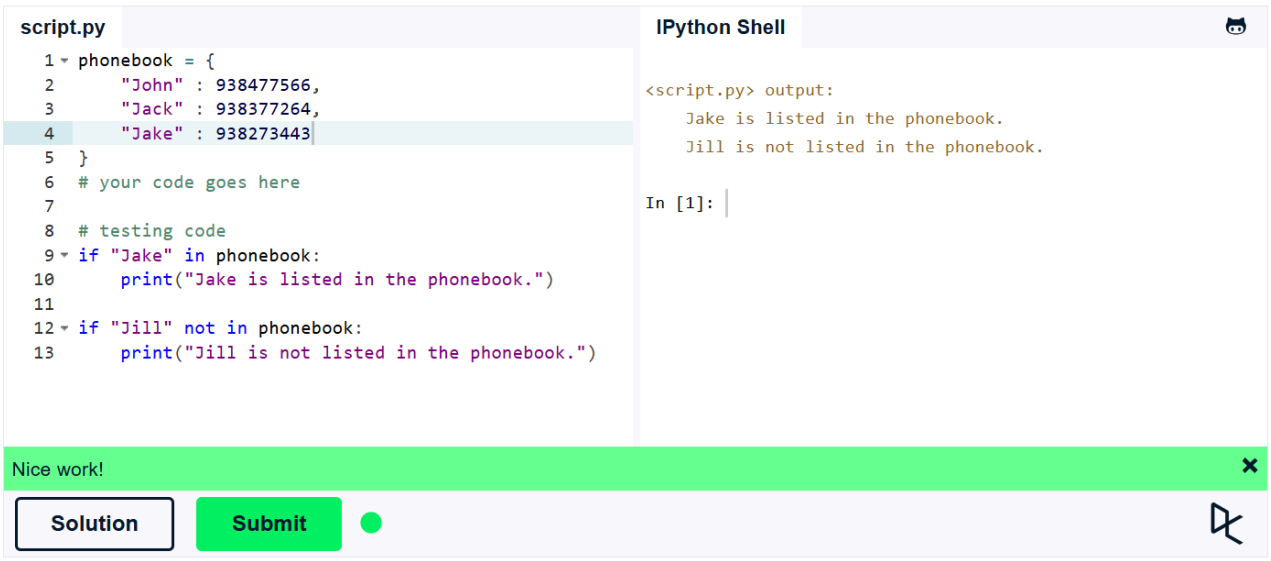


Figura 1.27 - Rezolvarea exercițiului cu dicționarul phonebook

Am adăugat persoana Jake împreună cu numărul său de telefon și am eliminat persoana Jill.

Module și pachete

Exercise

In this exercise, print an alphabetically sorted list of all the functions in the `re` module containing the word `find`.

script.py

```
import re

# Your code goes here
find_members = []
for member in dir(re):
    if "find" in member:
        find_members.append(member)

print(sorted(find_members))
```

IPython Shell

```
<script.py> output:
    ['findall', 'finditer']

In [1]:
```

Great work!

Solution Submit

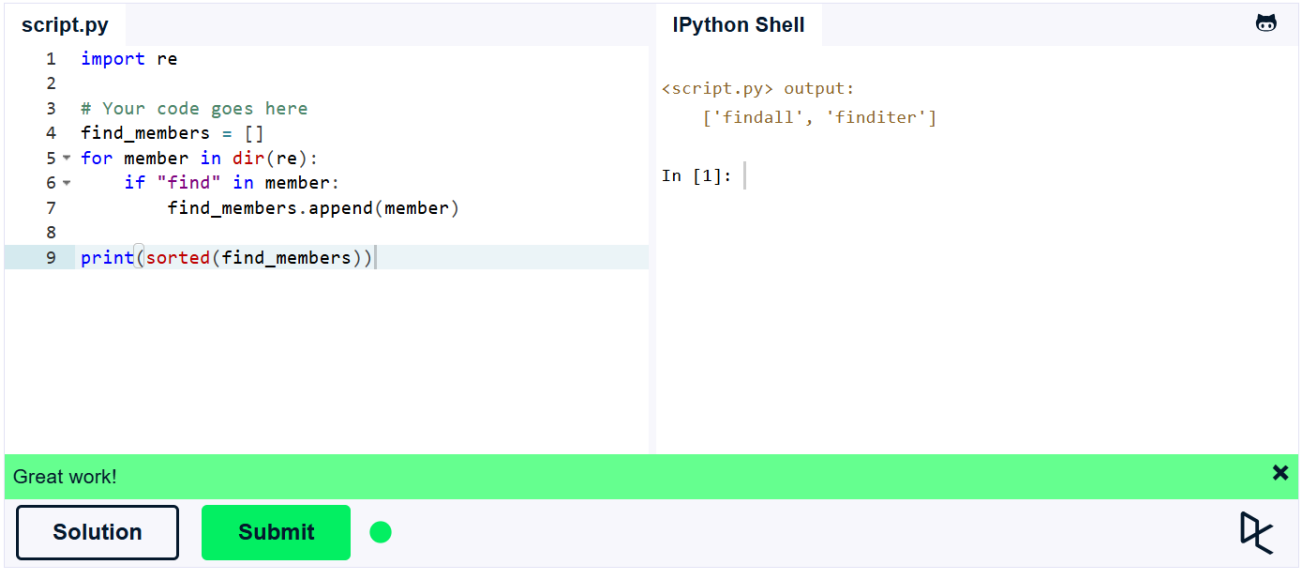


Figura 1.28 - Rezolvarea exercițiului cu modulul re și funcția dir()

Cu ajutorul funcției dir() am obținut elementele disponibile în acest modul. Am parcurs lista folosind o buclă for și am selectat elementele care conțin cuvântul find. Acestea au fost adăugate în lista find_members, iar cu ajutorul funcției sorted() au fost ordonate alfabetic și apoi afișate.

Citirea datelor – eroare

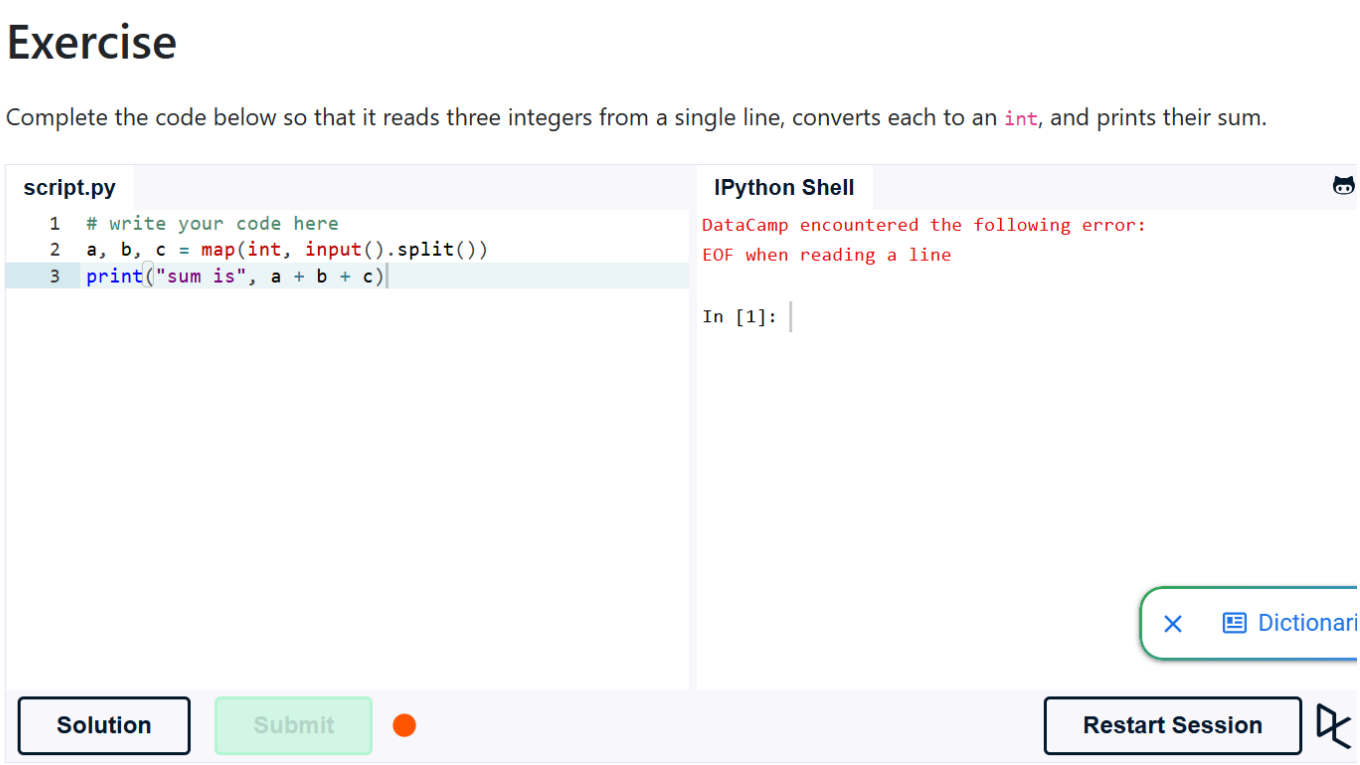


Figura 1.29 - Eroarea la rularea codului de citire a trei numere în editorul online

Citirea datelor – rezolvare

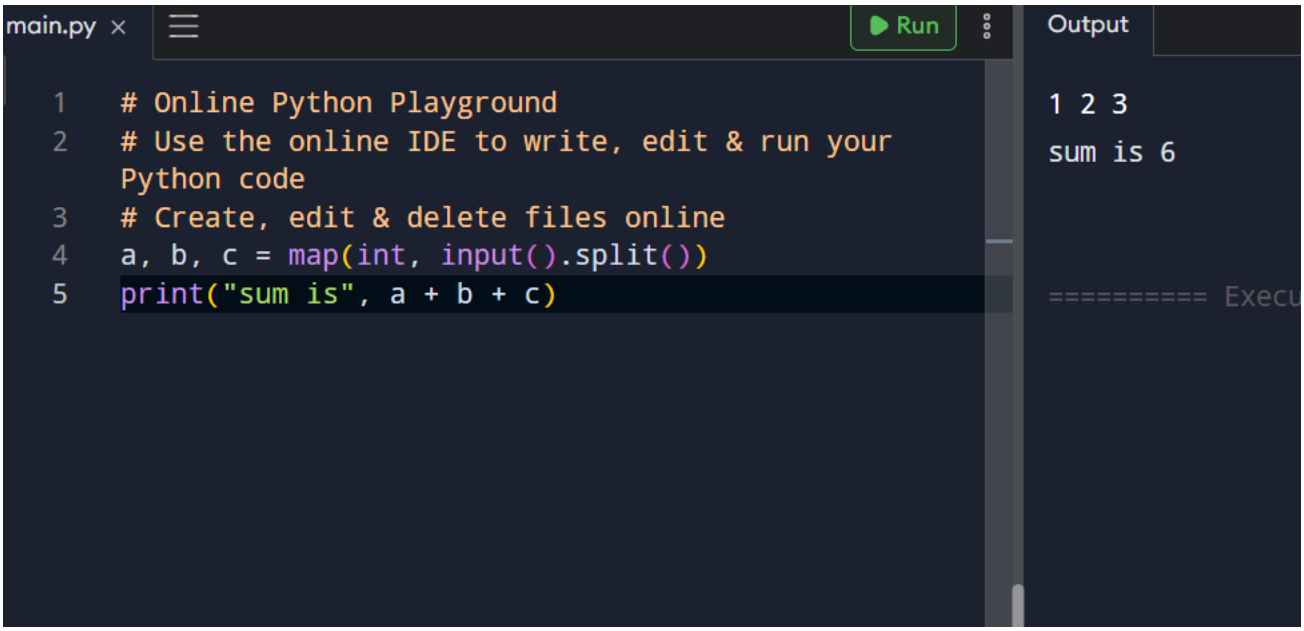


Figura 1.30 - Rezultatul rulării codului de citire a trei numere în Online Python Playground

Generatoare – enunț

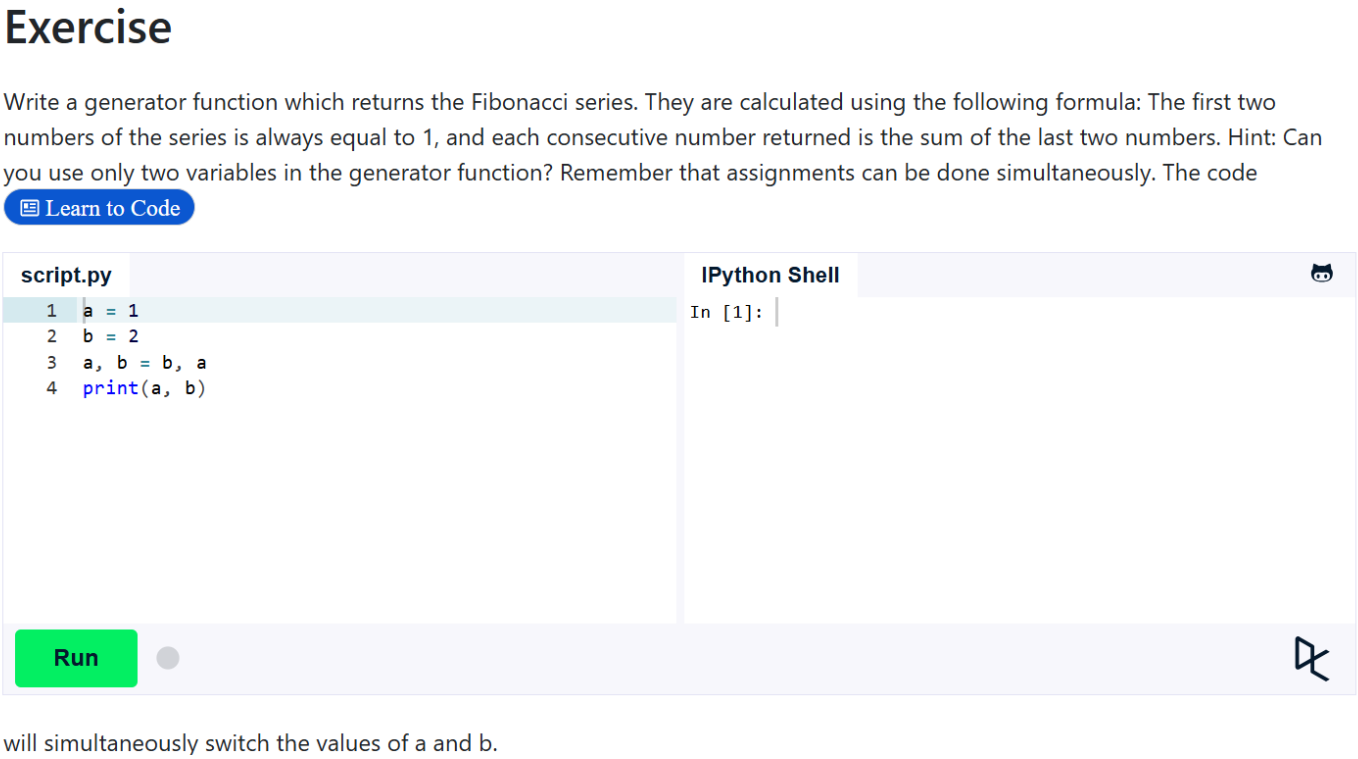


Figura 1.31 - Enunțul exercițiului și codul inițial pentru șirul Fibonacci

În acest exercițiu am utilizat funcția input() pentru a citi trei valori introduse pe aceeași linie. Metoda split() separă valorile introduse, iar funcția int() le transformă din șiruri de caractere în numere întregi. Cele trei numere sunt apoi adunate, iar rezultatul este afișat cu ajutorul funcției print().

Generatoare – rezolvare

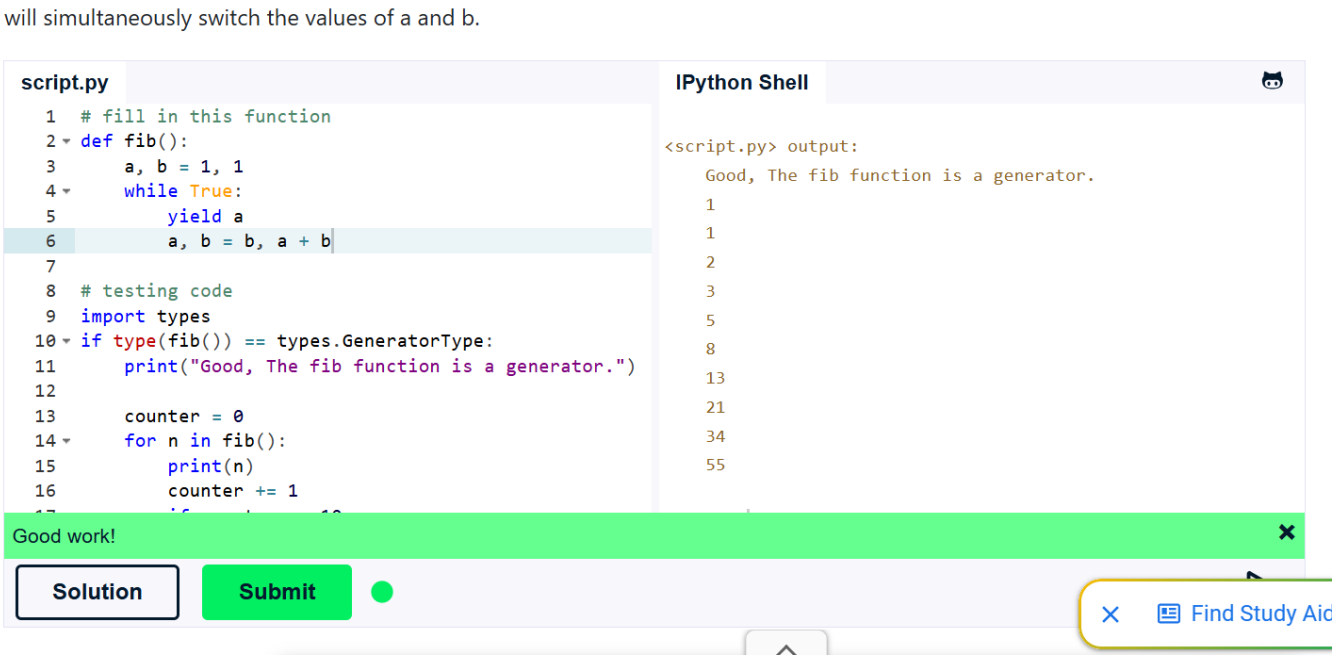


Figura 1.32 - Funcția generator pentru șirul Fibonacci

În acest exercițiu am creat o funcție generator pentru generarea șirului Fibonacci. Am inițializat primele două valori cu 1, iar cu ajutorul instrucțiunii yield am returnat succesiv fiecare număr din șir. Următoarele valori sunt calculate ca suma ultimelor două numere. Bucla while True permite generarea continuă a valorilor, iar codul de testare oprește execuția după afișarea primelor 10 numere din șirul Fibonacci.

List comprehension

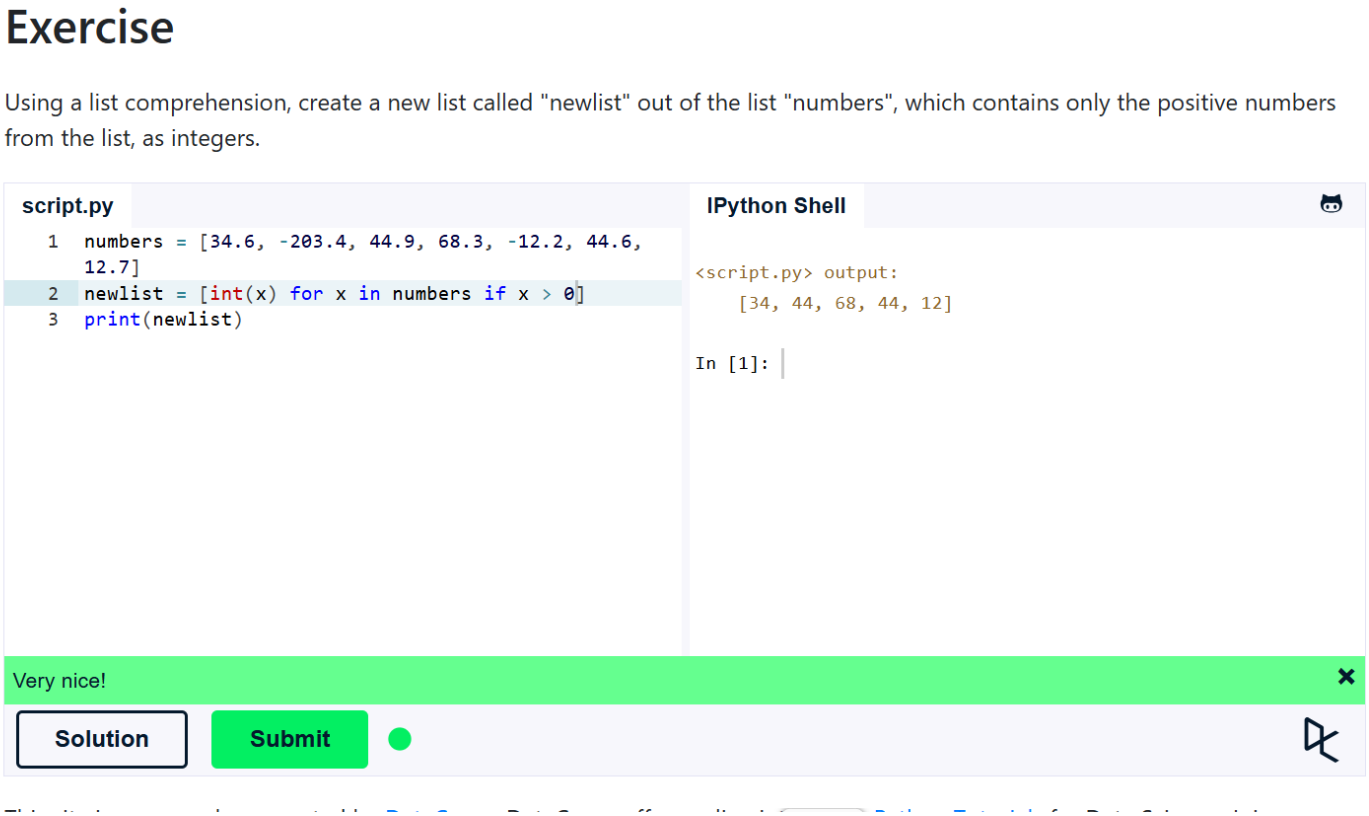


Figura 1.33 - Rezolvarea exercițiului cu list comprehension

În acest exercițiu am utilizat metoda „list comprehension” pentru a crea o listă nouă din lista inițială numbers. Am selectat doar valorile pozitive folosind condiția x > 0, iar cu funcția int() am transformat numerele zecimale în numere întregi. La final, lista obținută a fost afișată cu funcția print().

Funcții lambda

Exercise

Write a program using lambda functions to check if a number in the given list is odd. Print "True" if the number is odd or "False" if not for each element.

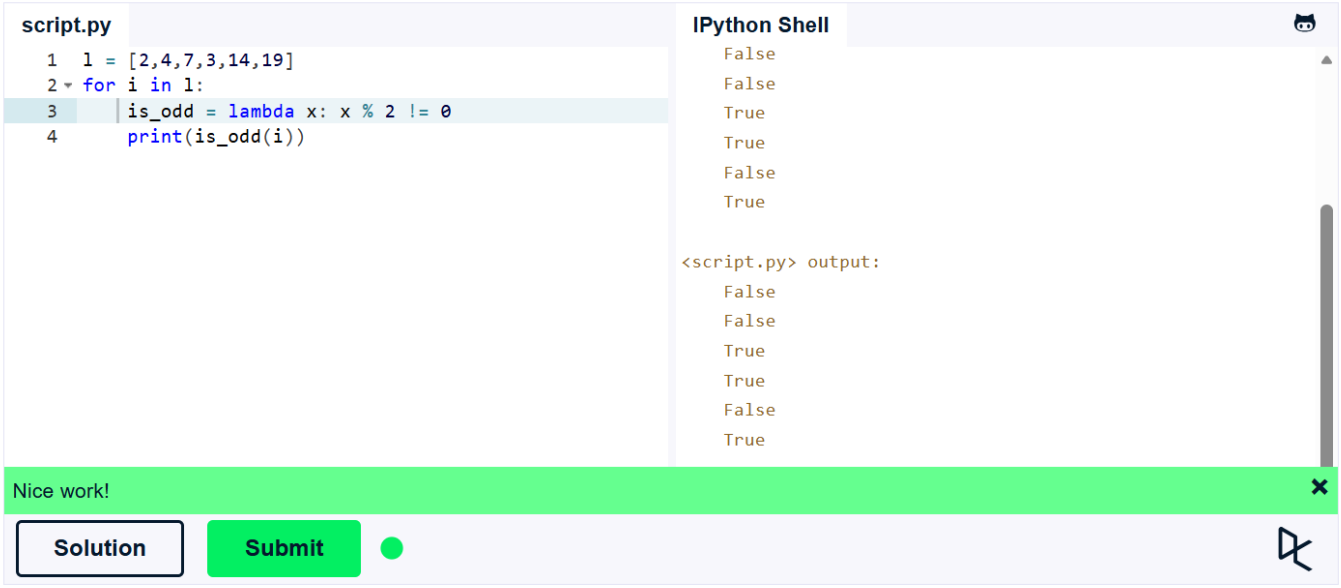


Figura 1.34 - Rezolvarea exercițiului cu funcția lambda pentru numere impare

În acest exercițiu am utilizat o funcție lambda pentru a verifica dacă numerele dintr-o listă sunt impare. Cu ajutorul operatorului % am verificat restul împărțirii fiecărui număr la 2. Dacă restul este diferit de 0, programul afișează True, iar dacă numărul este par, afișează False.

Argumente multiple

Exercise

Fill in the `foo` and `bar` functions so they can receive a variable amount of arguments (3 or more) The `foo` function must return the amount of extra arguments received. The `bar` must return `True` if the argument with the keyword `magicnumber` is worth 7, and `False` otherwise.

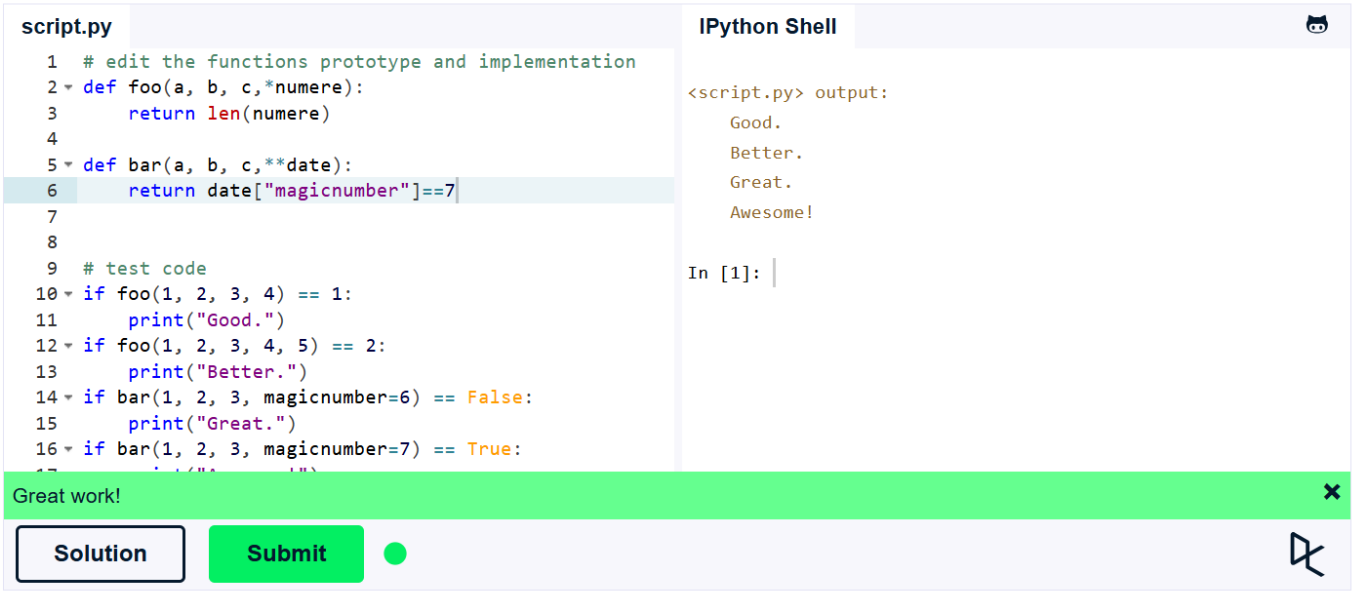


Figura 1.35 - Rezolvarea exercițiului cu *numere și **date

În acest exercițiu am utilizat *numere și **date pentru a permite funcțiilor să primească un număr variabil de argumente. Funcția foo() determină numărul argumentelor suplimentare primite, iar funcția bar() verifică dacă argumentul magicnumber are valoarea 7, returnând True sau False.

Expresii regulate

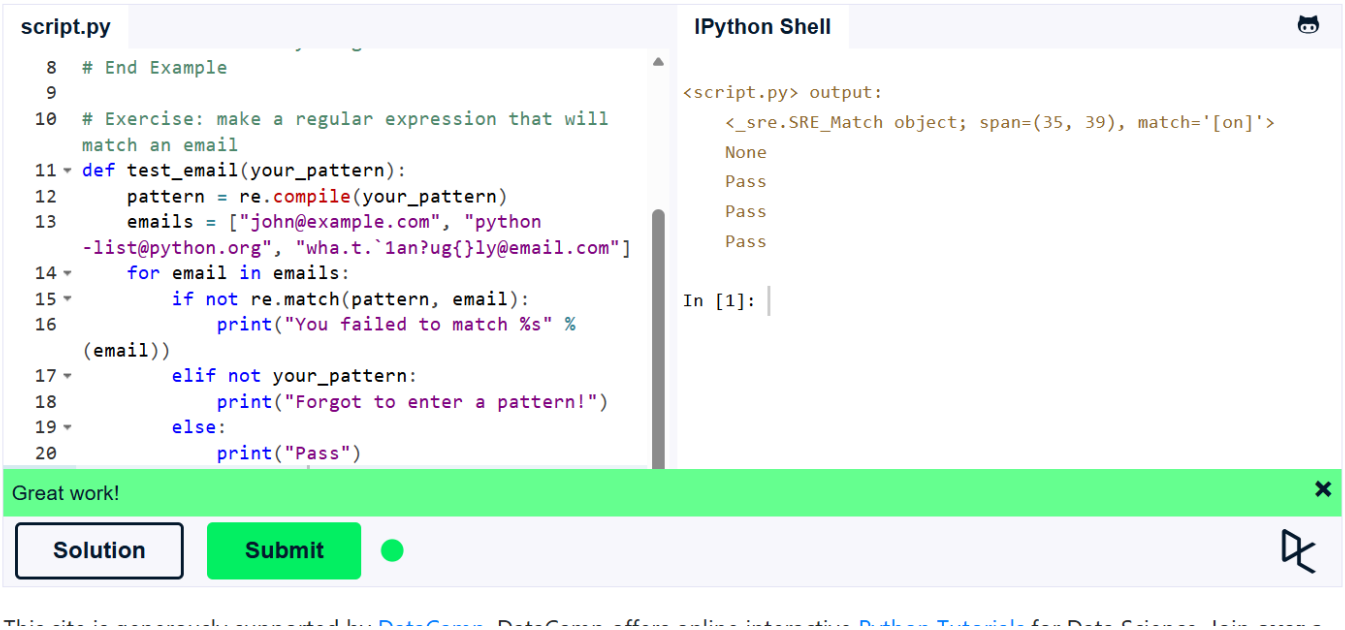


Figura 1.36 - Rezolvarea exercițiului cu expresii regulate pentru adrese de email

Am creat un pattern care stabilește structura pe care trebuie să o respecte adresa și am utilizat funcția test_email() pentru a verifica dacă adresele introduse corespund modelului stabilit.

Gestionarea excepțiilor

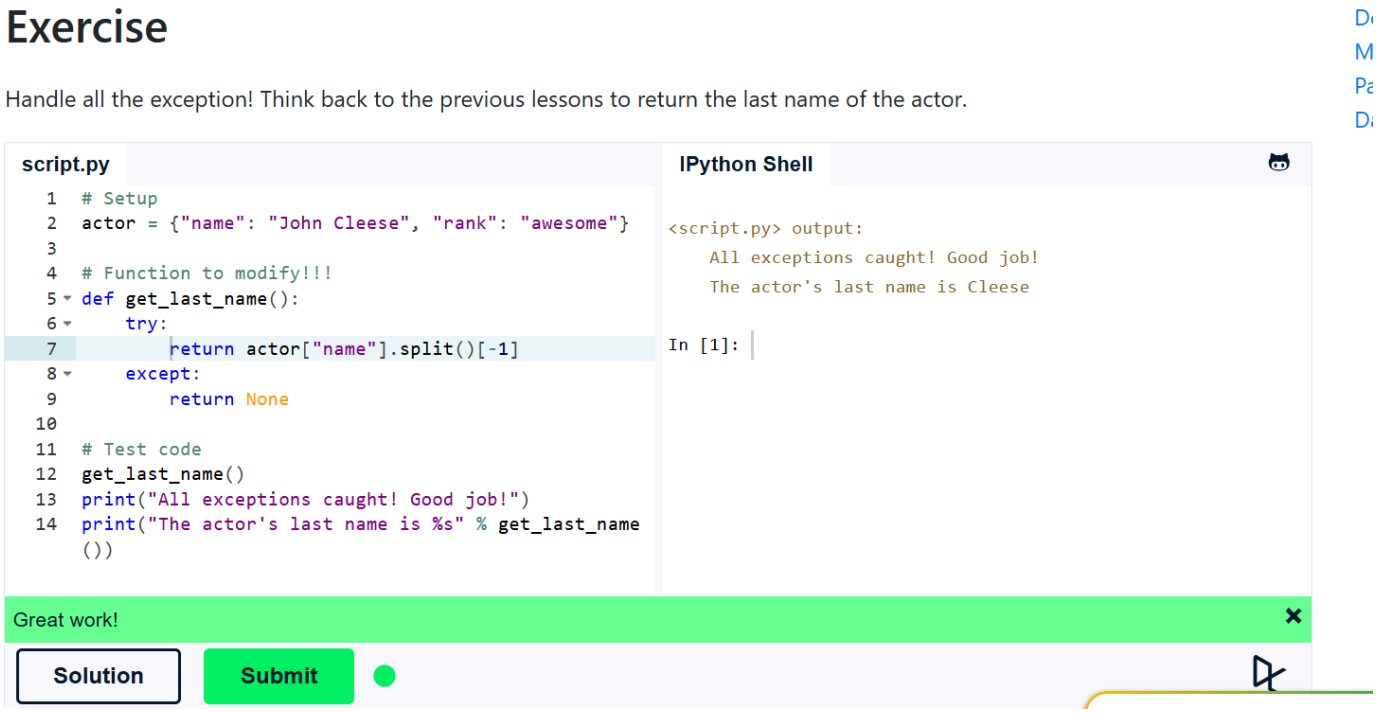


Figura 1.37 - Rezolvarea exercițiului cu try și except

În acest exercițiu am utilizat instrucțiunile try și except pentru gestionarea excepțiilor în Python. Am preluat numele actorului din dicționar, l-am separat în cuvinte și am returnat ultimul element, care reprezintă numele de familie.

Mulțimi

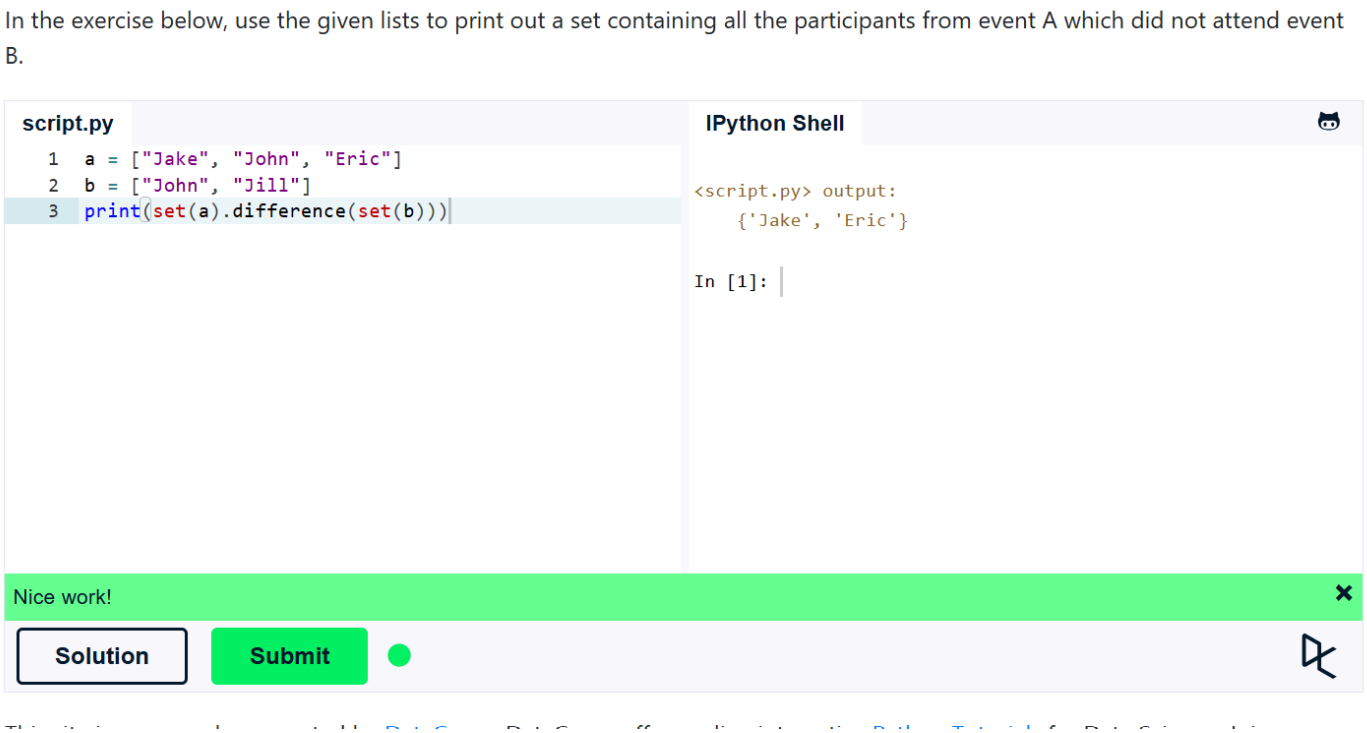


Figura 1.38 - Rezolvarea exercițiului cu mulțimi și operația difference()

În acest exercițiu am utilizat mulțimile (set) și operația difference() pentru a determina participanții care au fost prezenți la evenimentul A, dar nu au participat la evenimentul B.

Serializare JSON

The aim of this exercise is to print out the JSON string with key-value pair "Me" : 800 added to it.



Figura 1.39 - Rezolvarea exercițiului cu modulul json

În acest exercițiu am utilizat modulul json pentru serializarea și deserializarea datelor. Am transformat un șir JSON într-un dicționar Python, am adăugat o nouă pereche cheie-valoare și apoi am transformat datele înapoi în format JSON.

CONCLUZII

În cadrul lucrării de laborator am aprofundat utilizarea limbajului **Python** și am exersat mai multe concepte importante de programare, precum funcțiile, argumentele, funcțiile lambda, expresiile regulate, gestionarea excepțiilor, mulțimile și serializarea datelor în format JSON.

BIBLIOGRAFIE

1. Learn Python - Free Interactive Python Tutorial